



# Atelier Big Data

## Stockage Objet avec MinIO

Interface graphique et accès API avec Postman

**Objectif général :** découvrir MinIO comme solution de stockage objet compatible S3, manipuler les buckets et les objets à travers l'interface graphique, puis utiliser Postman pour appeler l'API S3 de MinIO.

Module : Big Data  
Niveau : Master II-BDCC et SDIA  
Année universitaire : 2025-2026

# 1 Présentation de l'atelier

## 1.1 Objectifs

À la fin de cet atelier, l'étudiant doit être capable de :

- expliquer le principe du stockage objet ;
- distinguer bucket, objet, clé d'objet et métadonnées ;
- lancer MinIO avec Docker Compose ;
- utiliser l'interface graphique de MinIO ;
- créer un bucket ;
- déposer, télécharger et supprimer des objets ;
- comprendre le rôle du port API et du port console ;
- configurer Postman pour accéder à l'API S3 de MinIO ;
- envoyer des requêtes API pour créer, lire, lister et supprimer des objets ;
- comparer l'approche MinIO avec HDFS.

## 1.2 Méthodes utilisées dans l'atelier

| Méthode                   | Rôle                                                                                                    |
|---------------------------|---------------------------------------------------------------------------------------------------------|
| Interface graphique MinIO | Créer des buckets, déposer des objets, observer les métadonnées, télécharger et supprimer des fichiers. |
| Postman                   | Tester l'API S3 de MinIO comme le ferait une application.                                               |
| API S3                    | Interface permettant aux applications d'envoyer, lire, lister et supprimer des objets.                  |

### Important

Dans cet atelier, nous n'utilisons ni le client `mc`, ni un programme Python. L'objectif est de comprendre MinIO avec l'interface graphique et avec des appels API depuis Postman.

# 2 Rappel théorique

## 2.1 Qu'est-ce que le stockage objet ?

Le stockage objet est une approche dans laquelle les données sont stockées sous forme d'objets.

Un objet contient généralement :

- les données elles-mêmes ;
- des métadonnées ;
- une clé unique permettant de l'identifier.

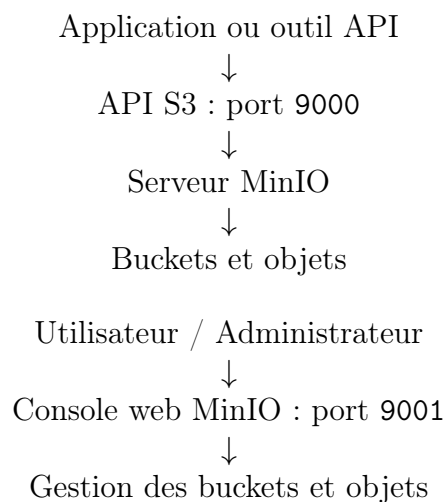
**Important**

Dans un système de fichiers classique, on pense en termes de dossiers et fichiers. Dans un stockage objet, on pense en termes de buckets, objets, clés et métadonnées.

## 2.2 Concepts fondamentaux

| Concept     | Description                                                                                           |
|-------------|-------------------------------------------------------------------------------------------------------|
| Bucket      | Conteneur logique qui stocke des objets.                                                              |
| Objet       | Donnée stockée dans un bucket : fichier CSV, image, log, PDF, modèle IA, etc.                         |
| Clé d'objet | Nom complet qui identifie un objet dans un bucket.<br>Exemple : <code>ventes/2026/ventes.csv</code> . |
| Métadonnées | Informations décrivant l'objet : taille, type, date, propriétaire, permissions.                       |
| API S3      | Interface permettant aux applications de communiquer avec le stockage objet.                          |

## 2.3 Architecture simple

**Important**

Le port 9001 sert à l'interface graphique. Le port 9000 sert à l'API utilisée par les applications et par Postman.

## 3 Préparation de l'environnement

### 3.1 Créer le dossier de travail

Créer un dossier nommé `atelier-minio`.

```
mkdir atelier-minio
cd atelier-minio
```

Créer le dossier de données :

```
mkdir data
```

L'architecture attendue est :

```
atelier-minio/  
  docker-compose.yml  
  data/  
    ventes.csv  
    clients.json  
    application.log  
    produit.txt
```

## 3.2 Créer les fichiers de données

Créer le fichier `data/ventes.csv` :

```
id,produit,ville,prix,quantite,date  
1,Ordinateur,Casablanca,8500,2,2026-05-01  
2,Souris,Rabat,120,10,2026-05-01  
3,Clavier,Marrakech,250,5,2026-05-02  
4,Ecran,Tanger,1800,3,2026-05-03  
5,Imprimante,Fes,2300,1,2026-05-04
```

Créer le fichier `data/clients.json` :

```
[  
  {"id": 1, "nom": "Ali", "ville": "Casablanca"},  
  {"id": 2, "nom": "Sara", "ville": "Rabat"},  
  {"id": 3, "nom": "Youssef", "ville": "Marrakech"}  
]
```

Créer le fichier `data/application.log` :

```
2026-05-01 10:15:22 INFO User login success  
2026-05-01 10:16:05 ERROR Payment service unavailable  
2026-05-01 10:17:41 INFO Order created  
2026-05-01 10:18:32 WARN Slow database query
```

Créer le fichier `data/produit.txt` :

```
Nom produit : Ordinateur portable  
Categorie : Informatique  
Prix : 8500 DH  
Stock : 25
```

## 4 Installation de MinIO avec Docker Compose

### 4.1 Créer le fichier `docker-compose.yml`

Dans le dossier `atelier-minio`, créer le fichier `docker-compose.yml` :

```
services:  
  minio:  
    image: minio/minio:latest
```

```

container_name: minio-server
ports:
  - "9000:9000"
  - "9001:9001"
environment:
  MINIO_ROOT_USER: admin
  MINIO_ROOT_PASSWORD: admin123
command: server /data --console-address ":9001"
volumes:
  - minio_data:/data

volumes:
  minio_data:

```

## 4.2 Explication du fichier Docker Compose

| Élément             | Rôle                                              |
|---------------------|---------------------------------------------------|
| minio               | Service qui lance le serveur MinIO.               |
| minio/minio:latest  | Image Docker de MinIO.                            |
| 9000:9000           | Port utilisé par l'API S3.                        |
| 9001:9001           | Port utilisé par la console web MinIO.            |
| MINIO_ROOT_USER     | Nom d'utilisateur administrateur.                 |
| MINIO_ROOT_PASSWORD | Mot de passe administrateur.                      |
| minio_data:/data    | Volume Docker utilisé pour conserver les données. |

## 4.3 Démarrer MinIO

Exécuter :

```
docker compose up -d
```

Vérifier que le conteneur fonctionne :

```
docker compose ps
```

Afficher les logs :

```
docker compose logs -f minio
```

### Travail à faire

Vérifier que MinIO est bien démarré et que les ports 9000 et 9001 sont exposés.

# 5 Manipulation de MinIO avec l'interface graphique

## 5.1 Accéder à la console web

Ouvrir le navigateur et accéder à :

<http://localhost:9001>

Utiliser les identifiants suivants :

- **Username** : minioadmin
- **Password** : minioadmin123

## 5.2 Créer un bucket

Créer un bucket nommé :

`stockage-cours`

### Travail à faire

Dans l'interface graphique de MinIO :

1. Cliquer sur **Buckets**.
2. Cliquer sur **Create Bucket**.
3. Saisir le nom `stockage-cours`.
4. Valider la création.

## 5.3 Ajouter des objets

Déposer les fichiers suivants dans le bucket `stockage-cours` :

- `ventes.csv`
- `clients.json`
- `application.log`
- `produit.txt`

### Travail à faire

Depuis la console web :

1. Ouvrir le bucket `stockage-cours`.
2. Cliquer sur **Upload**.
3. Sélectionner les fichiers du dossier `data`.
4. Vérifier que les fichiers apparaissent dans le bucket.

## 5.4 Observer les métadonnées

Pour chaque objet, observer :

- le nom de l'objet ;
- la taille ;
- le type de contenu ;
- la date de modification ;
- la clé de l'objet ;
- les informations d'accès.

## 5.5 Organiser les objets avec des préfixes

Créer l'organisation suivante :

```
stockage-cours/  
  ventes/  
    ventes.csv  
  clients/  
    clients.json  
  logs/  
    application.log  
  produits/  
    produit.txt
```

### Important

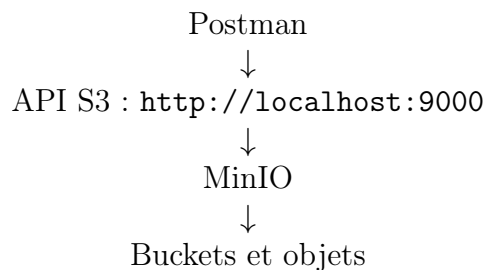
Les éléments comme `ventes/`, `clients/` et `logs/` ressemblent à des dossiers. Dans le stockage objet, ils représentent surtout des préfixes dans les clés des objets.

## 6 Préparation de Postman pour accéder à MinIO

### 6.1 Pourquoi utiliser Postman ?

Postman permet de tester les API sans écrire de code.

Dans cet atelier, Postman jouera le rôle d'une application qui communique avec MinIO.



### Important

L'interface graphique permet de manipuler MinIO visuellement. Postman permet de comprendre comment une application peut manipuler MinIO via l'API.

### 6.2 Créer une collection Postman

Dans Postman :

1. Créer une nouvelle collection.
2. Nommer la collection : `Atelier MinIO API`.
3. Ajouter les requêtes de l'atelier dans cette collection.

## 6.3 Configurer l'authentification AWS Signature

Pour chaque requête Postman, configurer l'onglet **Authorization** :

| Champ Postman | Valeur        |
|---------------|---------------|
| Type          | AWS Signature |
| Access Key    | minioadmin    |
| Secret Key    | minioadmin123 |
| AWS Region    | us-east-1     |
| Service Name  | s3            |

### Important

MinIO utilise une API compatible S3. Dans Postman, il faut donc utiliser l'authentification **AWS Signature** avec le service **s3**.

## 6.4 Créer des variables dans Postman

Créer les variables suivantes dans la collection :

| Variable  | Valeur                |
|-----------|-----------------------|
| endpoint  | http://localhost:9000 |
| bucket    | postman-bucket        |
| objectKey | ventes/ventes.csv     |

Ainsi, les URLs pourront être écrites sous la forme :

```
{{endpoint}}/{{bucket}}/{{objectKey}}
```

# 7 Manipulation de MinIO avec Postman

## 7.1 Requête 1 : Créer un bucket

|               |                         |
|---------------|-------------------------|
| Méthode       | PUT                     |
| URL           | {{endpoint}}/{{bucket}} |
| Authorization | AWS Signature           |
| Body          | Aucun                   |

### Travail à faire

Envoyer la requête, puis vérifier dans l'interface graphique MinIO que le bucket `postman-bucket` a été créé.

## 7.2 Requête 2 : Uploader un fichier dans un bucket

|         |                                           |
|---------|-------------------------------------------|
| Méthode | PUT                                       |
| URL     | {{endpoint}}/{{bucket}}/ventes/ventes.csv |



|                      |                                                               |
|----------------------|---------------------------------------------------------------|
| <b>Authorization</b> | AWS Signature                                                 |
| <b>Body</b>          | binary : sélectionner le fichier <code>data/ventes.csv</code> |

Dans Postman :

1. Aller dans l'onglet **Body**.
2. Choisir **binary**.
3. Sélectionner le fichier `ventes.csv`.
4. Envoyer la requête.

#### Travail à faire

Vérifier dans l'interface graphique MinIO que l'objet `ventes/ventes.csv` existe dans le bucket `postman-bucket`.

### 7.3 Requête 3 : Télécharger un objet

|                      |                                                        |
|----------------------|--------------------------------------------------------|
| <b>Méthode</b>       | GET                                                    |
| <b>URL</b>           | <code>{{endpoint}}/{{bucket}}/ventes/ventes.csv</code> |
| <b>Authorization</b> | AWS Signature                                          |
| <b>Body</b>          | Aucun                                                  |

#### Travail à faire

Envoyer la requête et observer le contenu du fichier dans la réponse Postman.

### 7.4 Requête 4 : Lister les objets d'un bucket

|                      |                                                  |
|----------------------|--------------------------------------------------|
| <b>Méthode</b>       | GET                                              |
| <b>URL</b>           | <code>{{endpoint}}/{{bucket}}?list-type=2</code> |
| <b>Authorization</b> | AWS Signature                                    |
| <b>Body</b>          | Aucun                                            |

La réponse est généralement au format XML.

#### Travail à faire

Envoyer la requête et identifier dans la réponse le nom de l'objet `ventes/ventes.csv`.

### 7.5 Requête 5 : Consulter les métadonnées d'un objet

|                      |                                                        |
|----------------------|--------------------------------------------------------|
| <b>Méthode</b>       | HEAD                                                   |
| <b>URL</b>           | <code>{{endpoint}}/{{bucket}}/ventes/ventes.csv</code> |
| <b>Authorization</b> | AWS Signature                                          |
| <b>Body</b>          | Aucun                                                  |

**Travail à faire**

Envoyer la requête et observer les headers de la réponse : taille, type de contenu, ETag et date de modification.

## 7.6 Requête 6 : Supprimer un objet

|                      |                                           |
|----------------------|-------------------------------------------|
| <b>Méthode</b>       | DELETE                                    |
| <b>URL</b>           | {{endpoint}}/{{bucket}}/ventes/ventes.csv |
| <b>Authorization</b> | AWS Signature                             |
| <b>Body</b>          | Aucun                                     |

**Travail à faire**

Envoyer la requête, puis vérifier dans l'interface graphique que l'objet a été supprimé.

## 7.7 Requête 7 : Supprimer un bucket

**Important**

Un bucket doit être vide avant d'être supprimé.

|                      |                         |
|----------------------|-------------------------|
| <b>Méthode</b>       | DELETE                  |
| <b>URL</b>           | {{endpoint}}/{{bucket}} |
| <b>Authorization</b> | AWS Signature           |
| <b>Body</b>          | Aucun                   |

**Travail à faire**

Supprimer le bucket uniquement après avoir supprimé tous les objets qu'il contient.

# 8 Exercice d'application avec Postman

## 8.1 Travail demandé

### Travail à faire

Créer un nouveau bucket nommé `entreprise-api-storage` avec Postman, puis réaliser les opérations suivantes :

1. Uploader le fichier `clients.json` avec la clé `clients/clients.json`.
2. Uploader le fichier `application.log` avec la clé `logs/application.log`.
3. Uploader le fichier `produit.txt` avec la clé `produits/produit.txt`.
4. Lister tous les objets du bucket.
5. Télécharger l'objet `clients/clients.json`.
6. Consulter les métadonnées de l'objet `logs/application.log`.
7. Supprimer l'objet `produits/produit.txt`.
8. Vérifier la suppression avec une requête de listing.

## 8.2 Organisation attendue

```
entreprise-api-storage/
  clients/
    clients.json
  logs/
    application.log
  produits/
    produit.txt
```

## 9 Comparaison MinIO vs HDFS

### 9.1 Tableau comparatif

| Critère                 | HDFS                          | MinIO                                  |
|-------------------------|-------------------------------|----------------------------------------|
| Modèle                  | Système de fichiers distribué | Stockage objet                         |
| Unité principale        | Fichier / bloc                | Objet                                  |
| Organisation            | Répertoires et fichiers       | Buckets et objets                      |
| Accès                   | Commandes HDFS                | Interface web, API S3                  |
| Écosystème historique   | Hadoop                        | Cloud, applications modernes, Big Data |
| Interface web           | Pas toujours intégrée         | Console web intégrée                   |
| Déploiement pédagogique | Plus lourd                    | Simple avec Docker                     |
| Utilisation applicative | Moins naturelle               | Très adaptée via API                   |

## 10 Travail de synthèse

## 10.1 Contexte

Une entreprise souhaite utiliser MinIO pour stocker différents types de fichiers :

- fichiers de ventes ;
- fichiers clients ;
- logs applicatifs ;
- documents produits ;
- fichiers divers.

L'objectif est de construire une organisation simple dans MinIO, puis de manipuler les objets avec Postman.

## 10.2 Travail demandé

### Travail à faire

Réaliser les tâches suivantes :

1. Créer un bucket nommé `mini-projet-storage`.
2. Organiser les objets avec les préfixes suivants :
  - `ventes/`
  - `clients/`
  - `logs/`
  - `produits/`
  - `documents/`
3. Déposer au moins un fichier dans chaque préfixe.
4. Lister tous les objets du bucket avec Postman.
5. Télécharger un objet avec Postman.
6. Consulter les métadonnées d'un objet avec Postman.
7. Supprimer un objet avec Postman.
8. Vérifier dans l'interface graphique que les opérations ont bien été effectuées.

## 10.3 Livrables

Les étudiants doivent rendre :

- une capture de la console MinIO montrant les buckets ;
- une capture montrant les objets organisés par préfixes ;
- une capture Postman pour chaque type de requête ;
- le fichier `docker-compose.yml` ;
- une courte réponse aux questions de compréhension ;
- une conclusion de 5 à 10 lignes.

## 11 Arrêt et nettoyage de l'environnement

### 11.1 Arrêter les conteneurs

```
docker compose down
```

### 11.2 Supprimer les conteneurs et le volume

Attention : cette commande supprime aussi les données stockées dans MinIO.

```
docker compose down -v
```

#### Important

La commande `docker compose down -v` supprime le volume Docker `minio_data`. Donc les buckets et objets créés pendant l'atelier seront supprimés.

## 12 Questions finales